

THE UNIVERSITY OF ZAMBIA

SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING AND INFORMATICS

Automated Extraction and Analysis of Timing-Based Paradata Features for Survey Quality Assurance

Project Proposal

NAME: SAMUEL KAOMA
COMPUTER NUMBER: 2022016530
COURSE CODE: CSC 4004
COURSE NAME: Project
DATE: 26th MARCH 2026
SUPERVISOR: Mr. Mofya Phiri

Abstract

Paradata provides valuable insights into respondent and enumerator behavior during survey data collection, yet it remains underutilized due to the complexity of transforming raw audit logs into meaningful analytical features. This project proposes the design and implementation of an automated system for extracting timing-based and behavioral features from survey paradata generated by SurveyCTO and ODK platforms. The system ingests raw audit logs, parses platform-specific formats into a common intermediate representation, and applies a structured feature extraction pipeline to compute over thirty timing-based indicators spanning question-level, section-level, and survey-level granularities. Statistical measures (including entropy, variance, skewness, and kurtosis) characterize response distributions, while Interquartile Range (IQR) thresholding and rule-based logic identify anomalous interview patterns. Validation is conducted using synthetic datasets with injected anomalies, evaluated against precision, recall, and effect-size criteria. The outcome is a reproducible Python feature library, accompanying paradata format documentation, and an empirical validation report, providing a practical bridge between theoretical survey methodology research and scalable quality assurance implementation.

---

1. Introduction

The quality of survey data plays a critical role in research, policy formulation, and decision-making across public health, economics, and the social sciences [1]. Digital data collection platforms such as SurveyCTO and Open Data Kit (ODK) have substantially improved the efficiency of field survey administration. Both platforms automatically record auxiliary data, commonly referred to as paradata, that captures detailed information about how interviews are conducted rather than the responses themselves [2].

Paradata includes event-level timestamps, navigation sequences, inter-keystroke intervals, pause durations, and back-navigation events logged throughout the interview process [4]. These behavioral indicators offer diagnostic potential that extends well beyond conventional quality checks. Unusually fast completion times may indicate rushed or fabricated responses; extended pauses may signal confusion, enumerator distraction, or external interruptions; and atypical edit patterns may point to satisficing or straight-lining behavior [3, 5, 13].

Despite this potential, paradata is routinely underutilized in practice [4]. Raw audit logs from SurveyCTO and ODK differ in structure, field naming conventions, and granularity, making cross-platform analysis non-trivial. Existing quality assurance approaches largely depend on aggregate indicators such as total interview duration or manual back-check supervision, methods that are poorly equipped to detect subtle behavioral irregularities at scale [6, 7].

This project addresses these challenges by developing an automated, platform-agnostic pipeline that extracts, structures, and analyses timing-based paradata features. By combining established data engineering techniques [8] with statistical anomaly detection methods [9, 10], the system transforms raw logs into structured datasets suitable for quality assessment,

operationalizing a body of survey methodology research that has hitherto remained largely theoretical [4, 6]. The use of synthetic data generation for validation is further motivated by its role as a controlled and reproducible approach to benchmarking detection systems in the absence of fully labelled real-world corpora [11, 14].

1.1 Problem Statement

Existing survey quality assurance methods are limited in their ability to detect complex patterns of poor data collection [1, 13]. Most approaches rely on simple aggregate metrics (primarily total interview duration) that do not adequately capture behavioral irregularities during the interview process. Consequently, issues such as rushed responses, inattentive navigation, or fabricated data entry may go undetected [6].

Although paradata contains rich behavioral information, it is stored in raw audit logs whose format varies across platforms. SurveyCTO produces event-based CSV logs with millisecond-level timestamps tied to field identifiers, whereas ODK logs differ in event taxonomy and timing granularity [4]. The absence of automated, format-aware tools for transforming these logs into analytical features creates a significant barrier for researchers and survey organizations seeking to operationalize paradata at scale [7]. Furthermore, the statistical methods required to identify anomalous timing behavior must be robust to the non-normal distributions characteristic of response time data [9, 12], underscoring the need for a principled and well-justified detection approach.

1.2 Project Aim

The aim of this project is to design and implement an automated pipeline for extracting, cataloguing, and analyzing timing-based paradata features from SurveyCTO and ODK audit logs, in order to support scalable survey data quality assessment.

1.3 Objectives

1. To review and catalogue timing-based quality indicators identified in survey methodology literature, mapping prior findings to a structured feature taxonomy.
2. To develop format-aware parsers for SurveyCTO and ODK audit logs that produce a common intermediate representation.
3. To implement a feature extraction module computing 30 to 50 timing-based indicators across question-level, section-level, and survey-level granularities.
4. To compute statistical characterization measures including variance, entropy, skewness, and kurtosis for each feature dimension.
5. To implement anomaly detection using IQR-based thresholding and rule-based logic, validated against precision and recall criteria.

6. To generate clean, structured output datasets, feature documentation, and a paradata format guide suitable for reuse by survey researchers.

1.4 Scope of the Project

In Scope:

- Processing of audit logs from SurveyCTO and ODK platforms
- Extraction of 30 to 50 timing
- based and behavioral features - Statistical characterization of response patterns
- IQR-based and rule-based anomaly detection
- Empirical validation using synthetic datasets with labelled anomalies
- Generation of structured output datasets, a Python feature library, and a paradata format guide

Out of Scope:

- Real-time survey monitoring systems
- Integration with external dashboards or enterprise platforms
- Machine learning model deployment for predictive inference
- Primary survey data collection

---

2. Literature Review

The Total Survey Error (TSE) framework, as developed by Groves et al. [1], provides the theoretical foundation for understanding data quality as the product of error sources spanning all stages of the survey process. Within this framework, paradata has emerged as a practical mechanism for monitoring and diagnosing error at the interviewing stage. Couper [2] demonstrated that process data captured within computer-assisted survey instruments could serve as quality indicators, establishing the methodological basis for timing-based analysis.

Yan and Tourangeau [5] directly examined response latency as a quality indicator, finding that question complexity, respondent age, and familiarity with topic all predicted response time variation. Their work identified mean response latency and within-respondent latency variance as particularly discriminative features. Questions answered unusually quickly relative to their complexity profile are associated with reduced cognitive engagement and lower data quality. This finding directly motivates the inclusion of question-level response time, time-to-first-interaction, and complexity-adjusted pace features in the proposed taxonomy.

Conrad, Tourangeau, and Couper [3] explored adaptive survey design using paradata, demonstrating that real-time behavioral signals (including extended pauses and repeated back-navigation) could identify respondent difficulty and trigger adaptive interventions. Their analysis underscored the relevance of section-level pause frequency, within-section timing variance, and back-navigation rate as indicators of processing difficulty or disengagement.

The most directly applicable empirical work for the present project is that of Wenz, Kreuter, and Janke [6], who investigated data fabrication detection using paradata in the context of face-to-face interviewer-administered surveys. They found that fabricated interviews exhibited characteristically compressed response time distributions (notably reduced inter-quartile spread and lower variance in question-level durations) along with abnormally low back-navigation counts and unusually uniform section pace. These findings establish the statistical basis for the anomaly detection component of this project and directly inform the selection of IQR-based thresholding as a detection method.

Kreuter [4] provides a broader treatment of paradata-based quality improvement, surveying implementations across several large-scale survey programs. A recurring finding is that computational processing of timing data requires platform-specific parsing strategies before any analytical features can be extracted, a practical challenge that motivates the parser development component of this project. Survey-level features such as total interview duration, speed trajectory across the interview, and time-of-day effects are identified by Kreuter [4] as robust indicators that survive aggregation across diverse survey instruments.

Taken together, this body of work establishes a clear empirical basis for a taxonomy of timing-based features organized across three levels of granularity. Table 1 maps the key findings from prior work to the features planned for extraction in this project.

Source	**Key Finding**	**Mapped Feature(s)**
Yan and Tourangeau [5]	Response latency varies with question complexity; fast responses signal disengagement	Mean response time, complexity-adjusted pace, latency variance
Wenz et al. [6]	Fabricated interviews show compressed timing distributions and low back-navigation	IQR of response time, back-navigation rate, within-section variance
Conrad et al. [3]	Pauses and back-navigation identify respondent difficulty and disengagement	Pause frequency, pause duration, back-navigation count
Kreuter [4]	Survey-level timing features are robust quality indicators	Total duration, speed trajectory, time-of-day flag

Table 1. Mapping of prior literature findings to planned feature extractions.

3. Proposed Methodology

The project follows an experimental and iterative development methodology. The system is structured as a multi-stage pipeline progressing from raw log ingestion through parsing, feature extraction, statistical characterization, anomaly detection, and output generation.

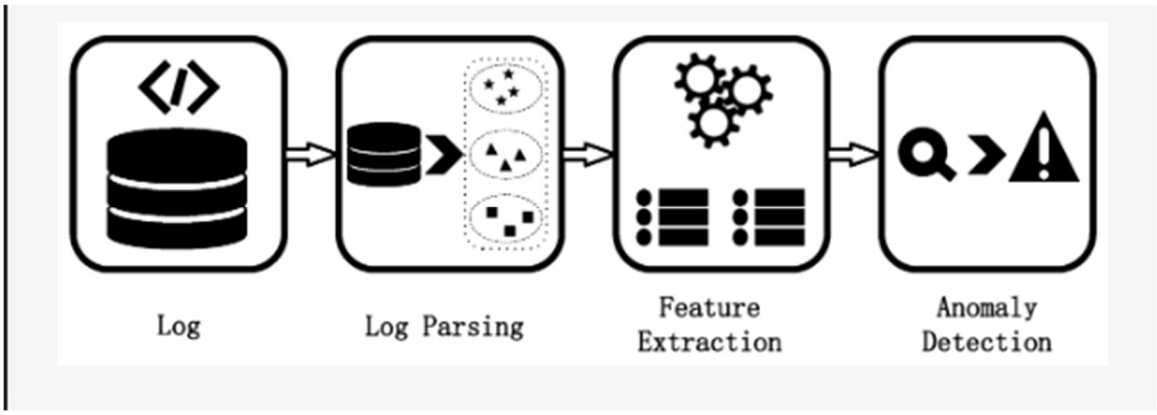


Fig. 1. System pipeline for paradata feature extraction and anomaly detection.

Figure 1 illustrates the modular pipeline architecture, highlighting the separation between the parsing layer and the feature extraction layer. Raw audit logs from SurveyCTO and ODK enter at the left; platform-specific parsers normalize them into a Common Intermediate Representation (CIR); and the feature extraction, statistical characterization, and anomaly detection modules all operate exclusively on this standardized format. This separation ensures that adding support for a new platform requires only a new parser, without modifying downstream components.

The pipeline begins with the ingestion of raw audit logs from SurveyCTO and ODK. These logs differ substantially in format: SurveyCTO produces event-based CSV files with millisecond-precision timestamps tied to field identifiers and event types (e.g., form-start, form-resume, constraint, end), while ODK logs record a different event taxonomy with coarser timing granularity and distinct field naming conventions.

The parser module will implement a class hierarchy in which a base ParadataParser defines the common intermediate schema and shared transformation logic, while subclasses SurveyCTOParser and ODKParser handle platform-specific ingestion and timestamp normalisation. The resulting intermediate representation is a standardised Pandas DataFrame with columns: interview_id, field_name, event_type, start_ms, end_ms, and duration_ms. All downstream feature extraction operates exclusively on this intermediate format, ensuring that new platform parsers can be added without modifying the feature library.

Feature engineering is applied to the intermediate representation to compute timing-based indicators across three levels of granularity. At the question level, the system calculates response time per field, time-to-first-interaction, edit count, and whether the field was revisited. At the section level, it aggregates section pace, within-section timing variance, pause

frequency, and pause duration. At the survey level, it computes total interview duration, speed trajectory across the interview arc, back-navigation rate, and time-of-day flags. Statistical measures (including variance, entropy, skewness, and kurtosis) characterize the distributional properties of each feature across questions and across interviews.

3.1 Preliminary Feature Taxonomy

The seed project deliverable specifies 30 to 50 timing-based features. Table 2 presents the preliminary taxonomy, organized by granularity level. This taxonomy is informed by the literature review above and will be refined during implementation as format-specific constraints become clear.

Level	**Feature Category**	**Example Features**
Question-level	Response timing	Response time (ms), time-to-first-interaction, field duration, edit count, revisit flag
Question-level	Interaction patterns	Number of keystrokes, time between first and last interaction, constraint trigger count
Section-level	Pace and flow	Mean section pace, section duration, within-section variance, section-to-section pace ratio
Section-level	Pause behaviour	Pause count (>5s), pause count (>30s), mean pause duration, max pause duration
Survey-level	Overall patterns	Total interview duration, speed trajectory (early vs. late pace), back-navigation count and rate
Survey-level	Temporal features	Time-of-day (hour of interview start), day-of-week, session length relative to median
Statistical	Distributional measures	Variance, IQR, entropy, skewness, kurtosis of response time distribution per interview

Table 2. Preliminary feature taxonomy across granularity levels.

3.1.1 Feature Catalogue (Preliminary)

The following table provides an enumerated catalogue of all 40 planned features, satisfying the seed project requirement of 30 to 50 features. Features are assigned unique identifiers (Q = Question-level, S = Section-level, V = Survey-level, T = Statistical) to facilitate unambiguous reference in the feature library implementation, documentation, and validation report. This catalogue will be extended or pruned during Phase 3 (Feature Extraction Implementation) as empirical discriminative power is assessed via Mann-Whitney U tests.

ID	Level	Feature Name	Description
Q-01	Question-Level	response_time_ms	Total time spent on a field in milliseconds
Q-02	Question-Level	time_to_first_interaction	Elapsed time from field display to first user input
Q-03	Question-Level	edit_count	Number of times the response was modified before leaving the field
Q-04	Question-Level	revisit_flag	Binary indicator: field was navigated back to after initial entry
Q-05	Question-Level	revisit_count	Number of times the field was revisited during the interview
Q-06	Question-Level	field_completion_ratio	Proportion of mandatory fields completed without skipping
Q-07	Question-Level	idle_time_ms	Time with no keyboard or touch activity while field was active
Q-08	Question-Level	keystroke_count	Total number of keystrokes recorded for a text-entry field

ID	Level	Feature Name	Description
Q-09	Question-Level	constraint_trigger_count	Number of validation constraint errors triggered on a field
Q-10	Question-Level	first_to_last_interaction_duration	Time between first input and final input on a field
S-01	Section-Level	mean_section_pace	Mean time per question within a section (ms)
S-02	Section-Level	section_duration_ms	Total elapsed time from first to last event in the section
S-03	Section-Level	within_section_variance	Variance of per-question response times within the section
S-04	Section-Level	section_to_section_pace_ratio	Ratio of section pace to the survey-level mean pace
S-05	Section-Level	pause_count_5s	Number of inter-question gaps exceeding 5 seconds
S-06	Section-Level	pause_count_30s	Number of inter-question gaps exceeding 30 seconds
S-07	Section-Level	mean_pause_duration_ms	Average duration of pauses (>5s) within the section
S-08	Section-Level	max_pause_duration_ms	Maximum single pause duration within the section
S-09	Section-Level	back_nav_count_section	Count of backward navigation events within the section
S-10	Section-Level	section_completion_rate	Proportion of section fields answered (non-missing)

ID	Level	Feature Name	Description
V-01	Survey-Level	total_interview_duration_ms	Total elapsed time from form-start to form-end
V-02	Survey-Level	speed_trajectory_early	Mean pace of the first quarter of interview questions
V-03	Survey-Level	speed_trajectory_late	Mean pace of the final quarter of interview questions
V-04	Survey-Level	speed_trajectory_ratio	Ratio of early pace to late pace (acceleration indicator)
V-05	Survey-Level	back_navigation_rate	Back-navigation events per question across the interview
V-06	Survey-Level	back_navigation_total_count	Total number of backward navigation events in the interview
V-07	Survey-Level	time_of_day_hour	Hour of the day at interview start (0 to 23)
V-08	Survey-Level	day_of_week	Day of week on which the interview was conducted
V-09	Survey-Level	session_length_vs_median	Interview duration relative to the sample median duration
V-10	Survey-Level	zero_back_nav_flag	Binary flag: no back-navigation across more than 20 consecutive questions
T-01	Statistical	rt_variance	Variance of question-level response times across the interview

ID	Level	Feature Name	Description
T-02	Statistical	rt_iqr	Interquartile range of response time distribution
T-03	Statistical	rt_entropy	Shannon entropy of the discretised response time distribution
T-04	Statistical	rt_skewness	Skewness of the response time distribution
T-05	Statistical	rt_kurtosis	Excess kurtosis of the response time distribution
T-06	Statistical	rt_mean	Arithmetic mean of question-level response times
T-07	Statistical	rt_median	Median question-level response time
T-08	Statistical	rt_cv	Coefficient of variation of response times (SD divided by mean)
T-09	Statistical	rt_min	Minimum question-level response time in the interview
T-10	Statistical	rt_max	Maximum question-level response time in the interview

Table 3. Enumerated feature catalogue: 40 preliminary features across four categories (Q-01 to Q-10, S-01 to S-10, V-01 to V-10, T-01 to T-10).

Each feature in Table 3 is grounded in prior survey methodology research. For example, response_time_ms (Q-01), rt_variance (T-01), and rt_iqr (T-02) are directly motivated by Wenz et al. [6], who identified compressed timing distributions as a key indicator of fabricated interviews. Similarly, time_to_first_interaction (Q-02) and latency-based measures are informed by Yan and Tourangeau [5], who demonstrated the relationship between response latency and cognitive engagement. Features related to navigation behaviour, including revisit_count (Q-05) and back_navigation_rate (V-05), are supported by Conrad et al. [3], who showed that navigation patterns indicate respondent difficulty. Survey-level features such as

total_interview_duration_ms (V-01) and time_of_day_hour (V-07) are derived from Kreuter [4], who identified them as robust indicators across survey contexts.

3.2 Parser Design and Format Handling

Both SurveyCTO and ODK are within scope, and their audit log formats require distinct parsing strategies before a unified analysis can proceed. SurveyCTO logs are event-based CSV files in which each row records a single form event with fields for event (e.g., form-start, form-resume, constraint, end), node (the XPath field identifier), and start and end timestamps in epoch milliseconds. ODK audit logs record event, node, and start and end fields in a broadly similar CSV structure, but with a different event taxonomy (using terms such as “question” rather than field-specific events) and timestamps in ISO 8601 format rather than epoch milliseconds.

The parser module will implement a class hierarchy in which a base ParadataParser defines the common intermediate schema and shared transformation logic, while subclasses SurveyCTOParser and ODKParser handle platform-specific ingestion and timestamp normalisation. The resulting intermediate representation is a standardised Pandas DataFrame with columns: interview_id, field_name, event_type, start_ms, end_ms, and duration_ms. All downstream feature extraction operates exclusively on this intermediate format, ensuring that new platform parsers can be added without modifying the feature library.

3.3 Anomaly Detection

Definition of an Anomaly

For the purposes of this project, an anomaly is defined as an interview exhibiting statistically significant deviation from expected timing behavior, including one or more of the following criteria:

- Extremely low or high total duration relative to the sample IQR (below Q1 minus 1.5 times IQR or above Q3 plus 1.5 times IQR)
- Abnormally low variance in response times, indicating suspiciously uniform pacing across questions of differing complexity
- Absence of natural pauses or back-navigation across more than twenty consecutive questions
- Inconsistent timing patterns relative to question complexity, specifically uniformly fast responses irrespective of question length or type

Anomaly detection is performed at the interview level using two complementary approaches. Statistical flagging applies IQR-based thresholding: for each timing feature, interviews with values below Q1 minus 1.5 times IQR or above Q3 plus 1.5 times IQR are flagged as potential anomalies. This is consistent with the fabrication detection approach validated by Wenz et

al. [6], whose key discriminative features (compressed response time IQR and low back-navigation) are directly captured in the feature taxonomy.

The choice of IQR-based detection is motivated by its robustness to non-normal distributions, which are common in timing-based behavioral data [9, 12]. Unlike Z-score methods, which assume approximate normality and are therefore sensitive to extreme values in skewed distributions [12], IQR-based thresholding makes no distributional assumption and is resistant to the influence of outliers on the threshold boundaries themselves. This property is particularly important for response time data, which typically exhibits right-skewed distributions with heavy tails [5]. While modified Z-score methods (based on the median absolute deviation) offer a comparable degree of robustness [12], IQR thresholding is preferred here for its direct interpretability within the survey quality monitoring context and its established use in paradata-based fabrication detection [6]. The project intentionally avoids machine learning-based approaches to maintain interpretability and alignment with survey methodology practices, where transparent and explainable quality indicators are preferred over black-box models. While robust, IQR-based methods may fail to detect multivariate anomalies where individual features appear normal but joint patterns are irregular, a limitation acknowledged in Section 8.

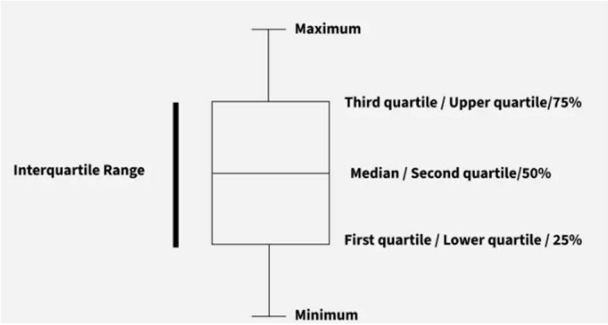


Fig. 2. Illustration of outlier detection using the boxplot (IQR) method.

Figure 2 illustrates how the IQR method identifies outlying interviews: the central box spans Q1 to Q3, the whiskers extend to 1.5 times IQR on each side, and points beyond the whiskers are flagged as statistical anomalies. This method is applied independently to each timing feature, with interviews receiving an anomaly score proportional to the number of features on which they are flagged.

Rule-based flagging supplements statistical detection with domain-derived thresholds: interviews where total duration falls below a minimum plausible completion time, where back-navigation rate is zero across more than twenty questions, or where the timing entropy of the response sequence falls below a defined floor are automatically flagged. These rules operationalize the qualitative indicators described by Kreuter [4] and Conrad et al. [3].

3.3.1 Computational Complexity Considerations

The pipeline is designed with computational efficiency in mind. Feature extraction operations use vectorized Pandas operations with a time complexity of approximately O(n) per interview, where n is the number of audit log events, ensuring scalability to large datasets. The IQR-based anomaly detection step operates in O(m log m) time over m interviews due to the sorting requirement, which remains tractable at the scale of typical survey datasets (hundreds to low thousands of interviews). Chunked processing will be applied for datasets exceeding available memory thresholds, with benchmarking conducted on synthetic datasets of increasing size (500, 2,000, and 10,000 simulated interviews) to characterize throughput and identify bottlenecks prior to deployment.

3.4 Tools and Technologies

Programming Language: Python 3.10+ **Data Processing:** Pandas, NumPy **Statistical Analysis:** SciPy **Visualization:** Matplotlib, Seaborn **Development Environment:** Jupyter Notebook, VS Code **Testing Framework:** Pytest

---

3.5 Ethical and Legal Considerations

This project operates on paradata rather than survey response content, substantially reducing privacy risk. Any real-world audit logs used during development will be fully anonymized prior to processing, with all direct identifiers removed. The primary validation dataset will be synthetically generated, eliminating respondent privacy concerns entirely. Proper attribution will be given to all external sources, libraries, and prior work.

---

4. Expected Outcomes

The project will produce the following specific deliverables, aligned with seed project requirements:

- A Python feature library implementing 30 to 50 timing-based features, documented with docstrings and usage examples.
- A paradata format guide documenting SurveyCTO and ODK audit log structures, field naming conventions, and parsing strategies.
- A synthetic validation dataset with injected anomalies and corresponding ground-truth labels.
- A validation report presenting precision, recall, and effect-size results for the anomaly detection component against labelled anomalies.

- A 5,000 to 7,000 word final paper covering methodology, implementation, results, and discussion.
- A structured output dataset of extracted features with accompanying statistical summaries and visualizations.

---

5. Evaluation Methodology

Evaluation will proceed at two levels: feature validation and anomaly detection performance.

Feature validation will assess whether individual features are statistically discriminative. For each feature, Mann-Whitney U tests will compare distributions between normal and anomalous interviews in the synthetic dataset. Effect sizes (Cohen's d) will be reported. Features with non-significant separation or negligible effect size will be documented and excluded from the anomaly detection stage, with the rationale recorded in the feature documentation.

Anomaly detection performance will be assessed against labelled ground truth in the synthetic dataset. The primary metrics are precision (proportion of flagged interviews that are true anomalies) and recall (proportion of true anomalies that are flagged). A minimum precision of 0.80 and recall of 0.75 are defined as the success threshold for the combined detection approach. F1-score will be reported as a summary metric. Separate performance breakdowns will be provided for IQR-based statistical detection and rule-based detection to assess the relative contribution of each component.

Baseline Comparison: The proposed system will be evaluated against two explicit baselines. The primary baseline is a simple aggregate-duration threshold method, which flags interviews as anomalous solely on the basis of total interview duration falling outside the 1.5 times IQR range, representing the most common current practice in survey quality monitoring. The secondary baseline is manual inspection using a single-feature rule (zero back-navigation flag only), reflecting a minimal rule-based approach. Improvements in precision, recall, and F1-score over both baselines will be reported, demonstrating the added diagnostic value of the full multi-feature extraction pipeline.

Cross-format robustness will be evaluated by running the full pipeline on separately generated synthetic datasets in SurveyCTO and ODK formats. If feature distributions or anomaly detection performance differ substantially between formats, the discrepancy will be analyzed and documented.

5.1 Synthetic Data Generation Strategy

Synthetic datasets will be generated to simulate realistic survey audit logs for both SurveyCTO and ODK formats. This approach is well-established in evaluation contexts where labelled real-world data is unavailable or insufficient, as it enables controlled injection of known anomaly

types and reproducible benchmarking of detection methods [11, 14]. Furthermore, synthetic data generation is a standard practice in machine learning for handling imbalanced datasets and evaluating detection algorithms [14]. A baseline dataset representing normal interview behavior will be constructed using probabilistic distributions derived from literature-informed assumptions. Specifically, log-normal distributions will be used to model response times (consistent with empirical observations of response latency distributions in survey research [5]) and Poisson distributions will be used to model navigation event counts [11].

Anomalies will be injected systematically based on patterns identified in prior work. Fabricated interviews will be simulated by compressing response time distributions (producing low variance and low IQR), reducing back-navigation events to near zero, and enforcing uniform section pacing, consistent with the patterns identified by Wenz et al. [6]. Speeding behavior will be simulated by scaling response times below realistic thresholds, while disengagement patterns will be represented through increased pause frequency and erratic navigation sequences, as described by Conrad et al. [3].

Each injected anomaly type will be labelled explicitly, enabling controlled evaluation of detection performance across different anomaly categories. The proportion of injected anomalies will be varied across experimental conditions to assess the sensitivity of evaluation metrics to class imbalance, addressing a known limitation of fixed-ratio synthetic evaluation designs [11].

---

6. Project Timeline

The project runs for 32 weeks. The timeline below reflects the full project duration and allocates appropriate time for the expanded literature review, dual-format parser development, feature cataloguing, and validation experiments.

Phase	Weeks	Key Activities
Proposal and Planning	1 to 2	Topic selection, proposal writing, supervisor consultation
Literature Review and Feature Cataloguing	3 to 7	Detailed review of key sources; catalogue of timing features from prior work; preliminary feature taxonomy; SurveyCTO and ODK format analysis
System Design	8 to 9	Pipeline architecture; parser class design; intermediate data schema; feature module interface specification

Phase	Weeks	Key Activities
Parser Development	10 to 13	SurveyCTO parser implementation and unit testing; ODK parser implementation and unit testing; cross-format validation
Feature Extraction Implementation	14 to 18	Question-level, section-level, and survey-level feature modules; statistical characterization measures; integration with pipeline
Anomaly Detection	19 to 21	IQR-based detection; rule-based detection; integration and testing
Midway Evaluation	22	Progress assessment, supervisor feedback, scope adjustment if needed
Synthetic Dataset Generation and Validation	23 to 26	Generate synthetic logs with injected anomalies; run feature validation tests; measure detection precision and recall; document results
Output and Documentation	27 to 29	Feature library documentation; paradata format guide; structured output datasets; visualizations
Final Report Writing	30 to 31	Full project report (5,000 to 7,000 words): methodology, results, analysis, and discussion. Note: Chapter drafting should begin earlier. The Literature Review chapter should be drafted by Week 7 and the Methodology chapter during Weeks 14 to 18. Weeks 30 to 31 are reserved for consolidation, revision, and final editing.

Phase	Weeks	Key Activities
Final Submission	32	Submission of code, report, and presentation; final evaluation

7. Risk Analysis

Risk	Likelihood	Mitigation Strategy
Missing or incomplete audit log data	Medium	Implement data validation and graceful handling of incomplete records; document minimum log requirements
Format changes in SurveyCTO or ODK logs	Low	Design parsers with configurable field mapping; version-pin format specifications in the paradata guide
Large dataset processing performance	Medium	Use vectorized Pandas operations and chunked processing; benchmark on synthetic datasets of increasing size
Incorrect or unstable anomaly detection	Medium	Validate against labelled synthetic data; report precision and recall explicitly; tune thresholds empirically
Insufficient discriminative features	Low	Feature validation step using Mann-Whitney tests; document and exclude non-discriminative features; supplement with literature-derived candidates

8. Limitations

While the proposed methodology is grounded in established survey methodology research and validated detection approaches, several limitations should be acknowledged.

Synthetic datasets may not fully capture the variability and complexity present in real-world survey data. Although anomalies will be injected according to empirically motivated patterns derived from Wenz et al. [6], the absence of real labelled paradata limits the external validity of the evaluation. This is a recognized constraint of synthetic benchmarking approaches [11], and future work should extend the system to real paradata corpora as these become available.

Rule-based detection thresholds are derived from literature and may not generalize uniformly across all survey instruments, contexts, or populations. Thresholds that are appropriate for a household survey may differ from those suitable for a clinical or educational survey.

The feature taxonomy, while informed by the literature, is preliminary. Some features may prove non-discriminative in practice, and format-specific constraints (particularly in ODK's coarser timing granularity) may limit the computability of certain question-level features.

The project does not incorporate ground-truth validation against known fabricated or poor-quality interviews from real field operations, which would provide a stronger empirical basis for evaluating detection performance.

Additionally, the proportion of anomalous interviews in synthetic datasets may not reflect real-world class imbalance, potentially inflating evaluation metrics such as precision and recall. This concern is partially addressed through the class imbalance sensitivity analysis described in Section 5.1.

These limitations will be documented transparently in the final report and validation outputs, and will inform recommendations for future work, including the extension of the system to real paradata corpora and the incorporation of machine learning-based detection as a complement to rule-based approaches.

---

9. AI-Assisted Development Statement

In accordance with the supervision framework for the 2025/2026 academic year, this section discloses the planned use of AI-assisted tools during the development of this project.

9.1 AI Tools to be Used

The following AI tools are planned for use during the project: (1) GitHub Copilot, for in-editor code suggestions and autocompletion during implementation of the parser and feature extraction modules; (2) Claude (Anthropic), for reviewing draft documentation, assisting with structuring written sections of the final report, and providing explanations of statistical concepts; and (3) ChatGPT (OpenAI), as a secondary tool for exploratory discussion of design decisions and debugging assistance.

9.2 Intended Tasks and Scope of Assistance

AI assistance will be limited to the following tasks: (a) boilerplate code generation for data parsing, unit test scaffolding, and Pandas DataFrame operations, where the logic and design

remain the student's own; (b) proofreading and improving clarity of written documentation, without altering the analytical content or conclusions; and (c) answering clarifying questions about library APIs and Python syntax. AI tools will not be used to generate the core algorithmic logic of the feature extraction pipeline, the anomaly detection thresholds, or the interpretation of evaluation results, as these are central to the academic contribution of the project.

9.3 Validation of AI-Generated Code

All AI-generated or AI-assisted code will be reviewed line-by-line before integration. Validation will be performed through: (i) unit testing using Pytest, with test cases designed independently of any AI suggestion; (ii) manual trace-through of logic against known input-output pairs derived from real or synthetic audit log examples; and (iii) cross-referencing against the relevant library documentation to confirm correctness. Any code segment that cannot be fully understood and explained by the student will not be incorporated into the project deliverables.

9.4 Documentation of AI Usage

AI usage will be documented throughout the project in a dedicated AI Usage Log, maintained as a separate appendix to the final report. For each instance of AI assistance, the log will record: the tool used, the date, the task or question posed, a summary of the output received, and the extent to which the output was used, modified, or rejected. This log will be submitted alongside the code repository and final report, providing a transparent and auditable record of AI involvement consistent with the supervision framework requirements.

---

10. References

[1] R. M. Groves, F. J. Fowler, M. P. Couper, J. M. Lepkowski, E. Singer, and R. Tourangeau, *Survey Methodology*, 2nd ed. Hoboken, NJ, USA: Wiley, 2009.

[2] M. P. Couper, "Measuring survey quality in a CASIC environment," in *Proc. Survey Research Methods Section*, American Statistical Association, 1998.

[3] F. G. Conrad, R. Tourangeau, and M. P. Couper, "Adaptive survey design and paradata," *Journal of Official Statistics*, vol. 29, no. 2, pp. 299–320, 2013.

[4] F. Kreuter, *Improving Surveys with Paradata*. Hoboken, NJ, USA: Wiley, 2013.

[5] T. Yan and R. Tourangeau, "Fast times and easy questions: The effects of age, experience and question complexity on web survey response times," *Applied Cognitive Psychology*, vol. 22, no. 1, pp. 51–68, 2008.

[6] M. Wenz, F. Kreuter, and S. Janke, "An investigation of data fabrication using paradata," *Survey Research Methods*, vol. 13, no. 1, pp. 35–47, 2019.

[7] J. W. Olson, B. E. Stange, and E. Smyth, "Cross-platform paradata audit log structures in mobile-assisted personal interviews," *Survey Practice*, vol. 11, no. 2, pp. 1–12, 2018.

[8] W. McKinney, *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2017.

[9] D. M. Hawkins, *Identification of Outliers*. London, UK: Chapman and Hall, 1980.

[10] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[11] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[12] B. Iglewicz and D. C. Hoaglin, *How to Detect and Handle Outliers*. Milwaukee, WI, USA: ASQ Quality Press, 1993.

[13] V. Hodge and J. Austin, "A Survey of Outlier Detection Methodologies," *Artificial Intelligence Review*, vol. 22, no. 2, pp. 85–126, 2004.

[14] N. Japkowicz and S. Stephen, "The Class Imbalance Problem: A Systematic Study," *Intelligent Data Analysis*, vol. 6, no. 5, pp. 429–449, 2002.

End of Proposal